Tutoriel : création d'une application iOS avec une UITableView

Les `UITableView` sont un peu le couteau suisse des applications iOS. La plupart du temps, les premières applications sont construites avec ou autour de ces *Views* extrêmement pratiques et facilement améliorables.

Ce tutoriel est très fortement inspiré de celui proposé par *BN* sur https://medium.com/ios-world/simple-uitableview-application-in-objective-c-becc0f6f2e5e.

Création du projet Xcode

Lancez Xcode (nous supposons que nous travaillons avec macOS Monterey 12.7.6 et Xcode 14.2).

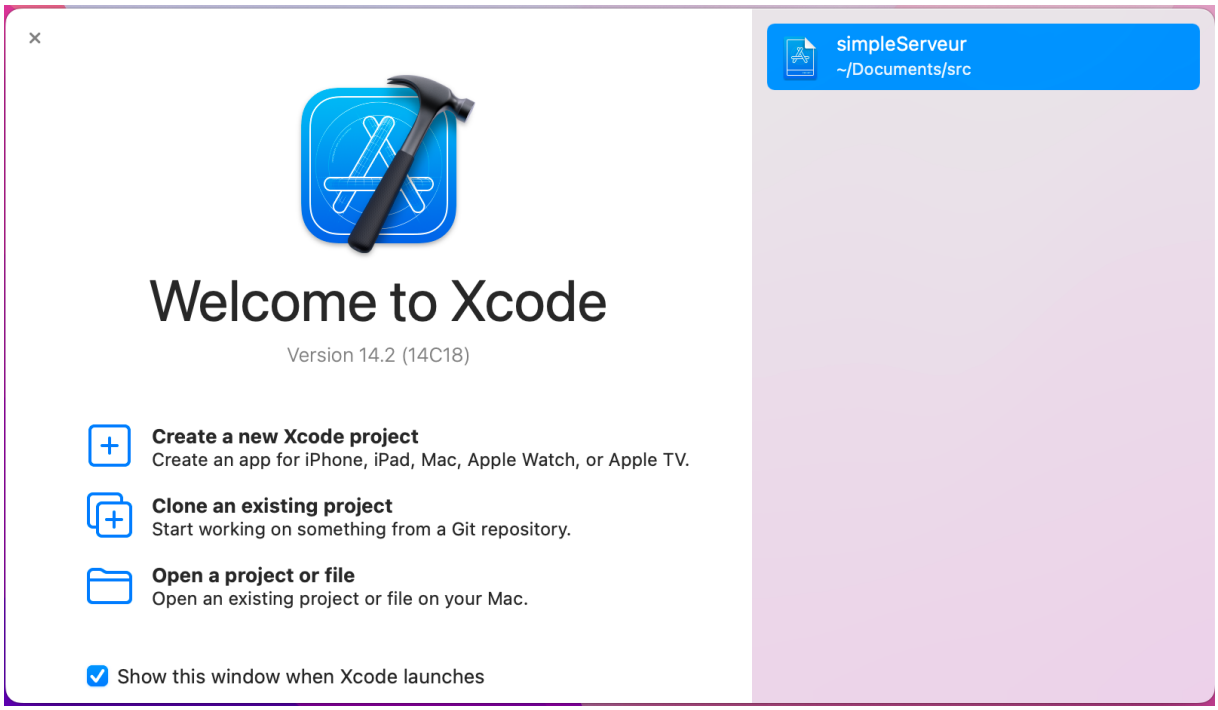


Figure 1Démarrage d'Xcode.

Au démarrage d'XCode choisissez `Create a new Xcode project`. Ou, si Xcode était déjà démarré, faites : `File > New > Project`…

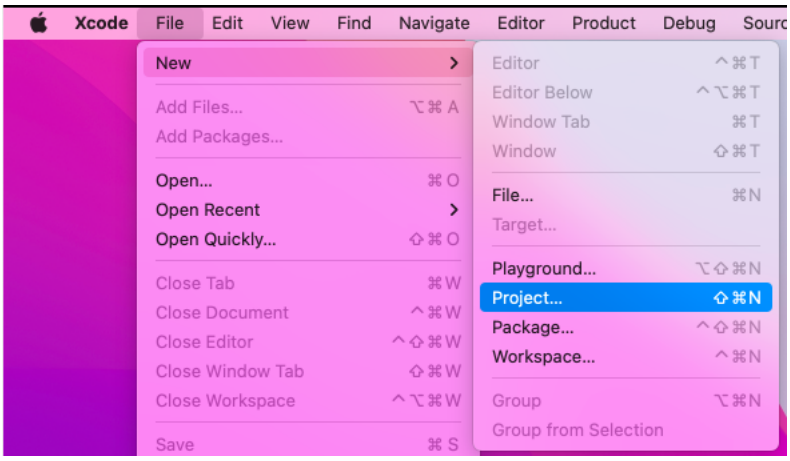


Figure 2Créer un nouveau projet.

Choisissez le modèle (*template*) App pour `iOS` :

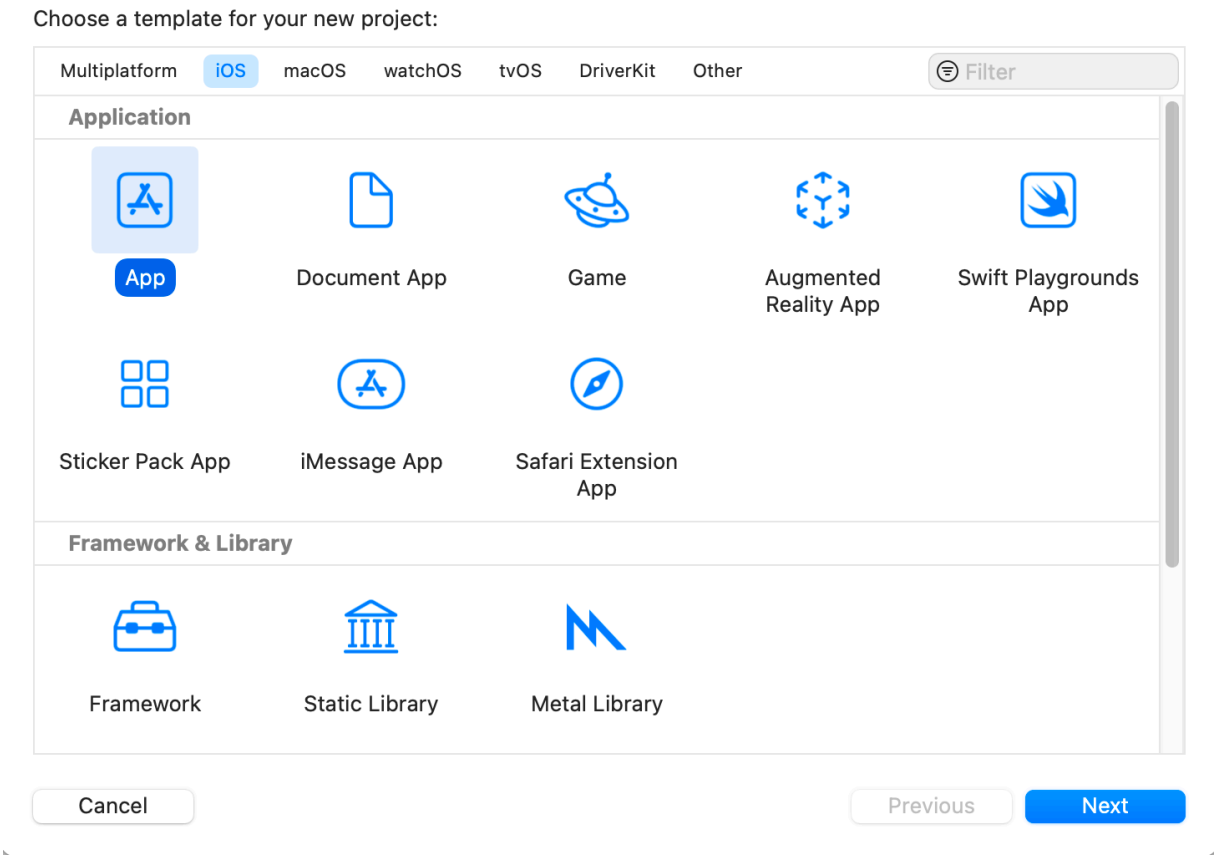


Figure 3Template iOS App.

cliquez sur `Next`, donnez un nom à votre application (ici `TableView Tuto`) et surtout sélectionnez `Interface : Storyboard` et `Language : Objective-C`.

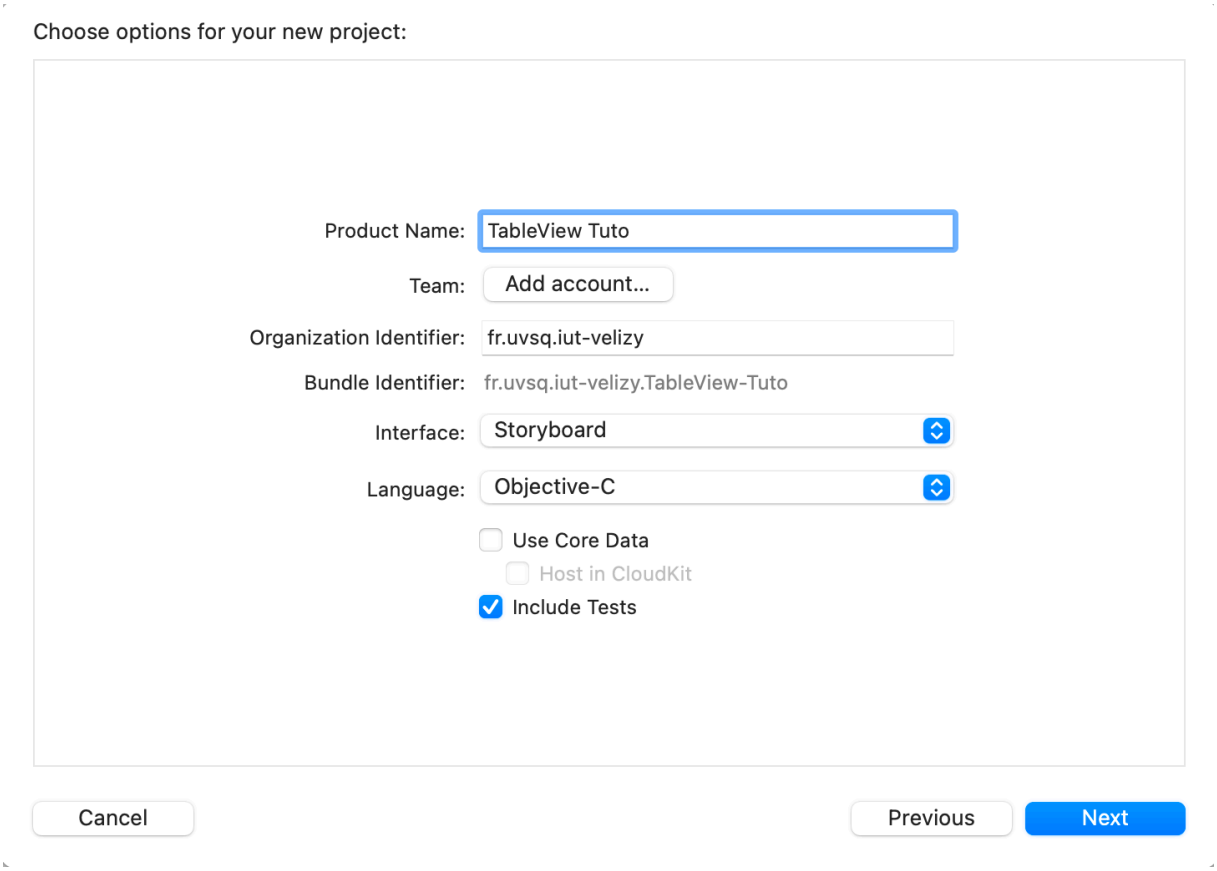


Figure 4Choisir les paramètres du projet (nom, interface et langage).

Choisissez ensuite où stocker le projet (je recommande `~/Documents/src/`) :

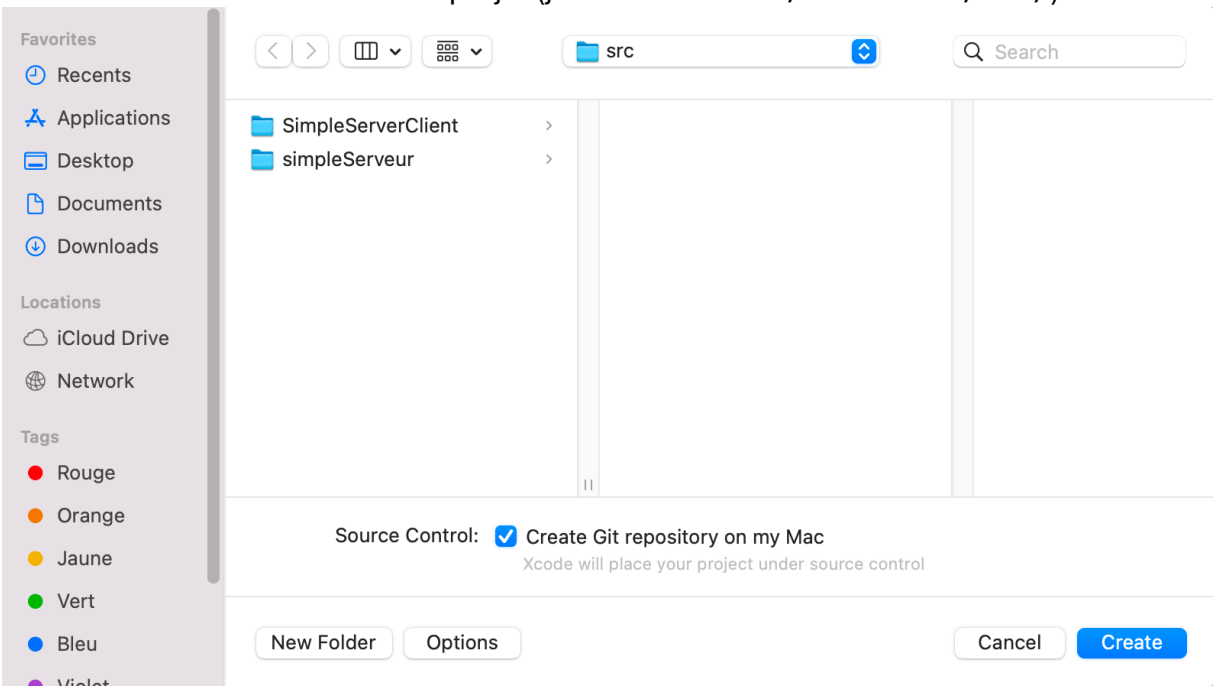


Figure 5Localisation des sources.

Cliquez sur `Create`. Vous arrivez alors dans l'environnement de votre projet.

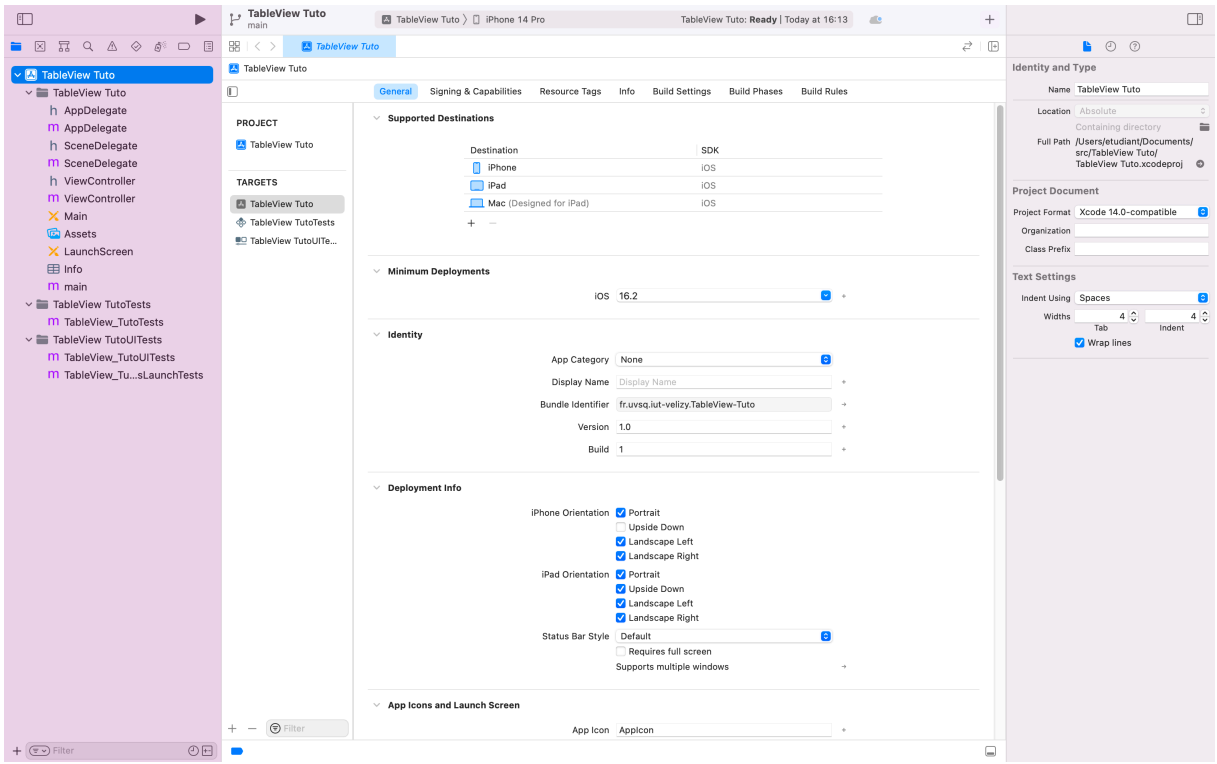


Figure 6Environnement de conception de notre projet.

Création de l'interface

Nous allons maintenant placer les *widgets* (c'est-à-dire les `Views`) qui nous interessent sur l'écran de notre application.

A gauche de l'écran cliquez sur le `Main.storyboard` (l'extension `storyboard` n'apparait pas dans l'interface, c'est l'icone orange avec un crayon et un pinceau qui forment un X). Vous verrez apparaître la silhouette d'un iPhone (par défaut l'iPhone 14 Pro, je préfère sélectionner le plus petit format, c'est-à-dire l'iPhone SE 3rd gen, pour des raisons de taille d'écran de mon environnement de développement). Cliquez ensuite sur le + en à gauche de l'écran pour faire apparaitre la bibliothèque de *widgets* que vous pouvez ajouter. Tapez table dans le champ Objects pour filtrer la recherche.

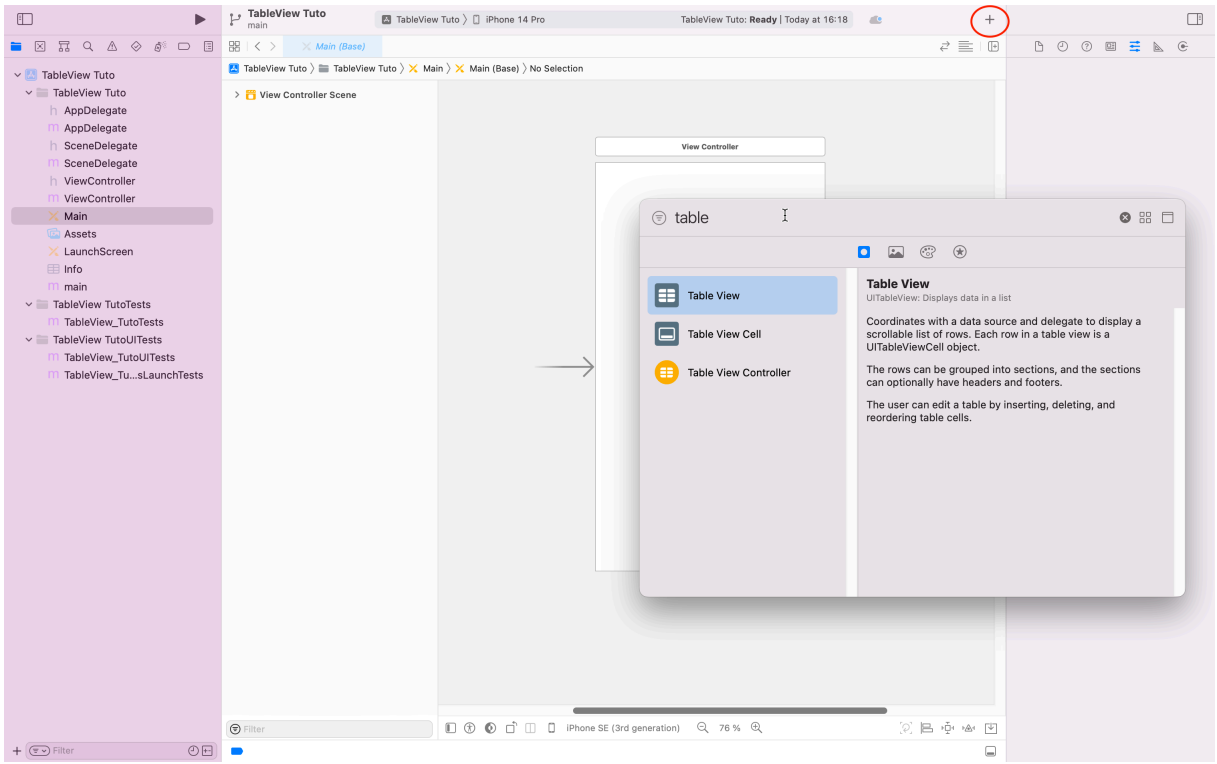


Figure 7 Sélection des widgets à insérer.

Sélectionnez `Table View` et faites-le glisser dans le `ViewController` (la silhouette du téléphone dans le `Main.storyboard`).

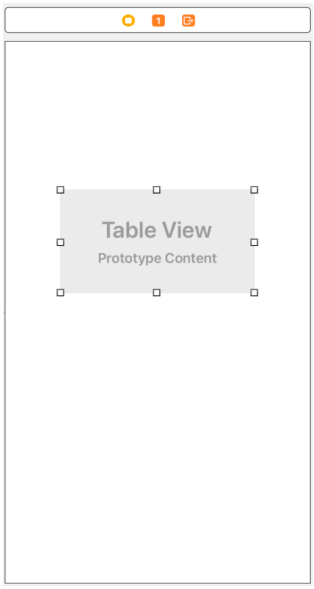


Figure 8On place le Table View sur l'écran.

Ajustez la taille du `Table View` pour qu'il occupe tout l'écran sauf un espace en bas (on y placera plus tard un bouton). Et ajoutez les contraintes suggérées.

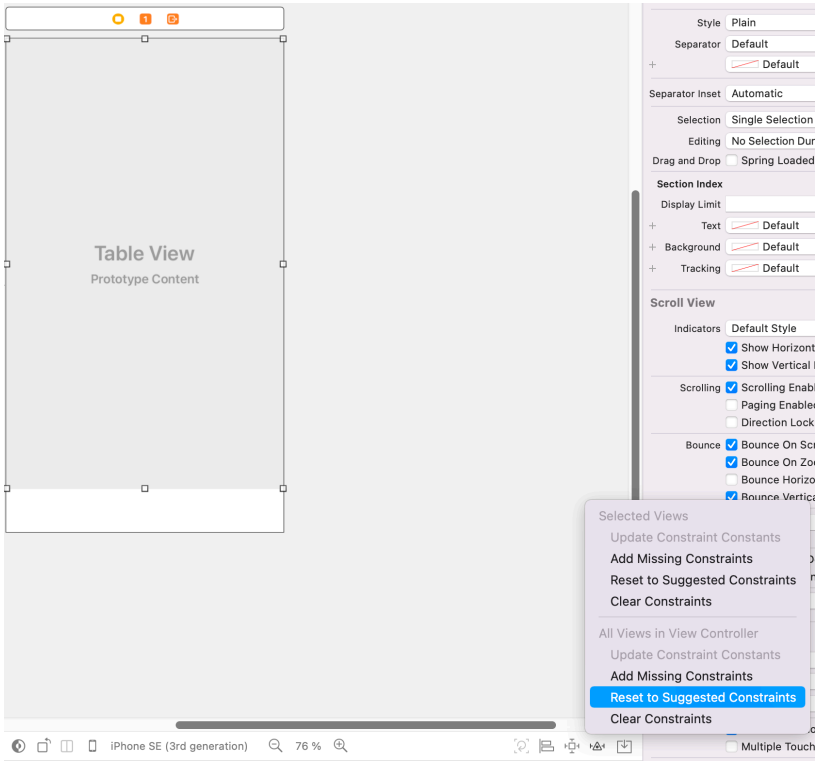


Figure 9Le Table View occupe presque tout l'écran, on ajoute les contraintes.

Opération très importante : sélectionnez votre `Table View`, maintenez la touche `contrôle` appuyez, cliquez sur le `Table View` et glissez la souris jusqu'à l'icône orange (un disque orange avec un carré blanc à l'intérieur ; elle représente le `View Controller`) en haut à gauche du téléphone, sélectionnez `Data Source` ; recommencez et sélectionnez `Delegate`.

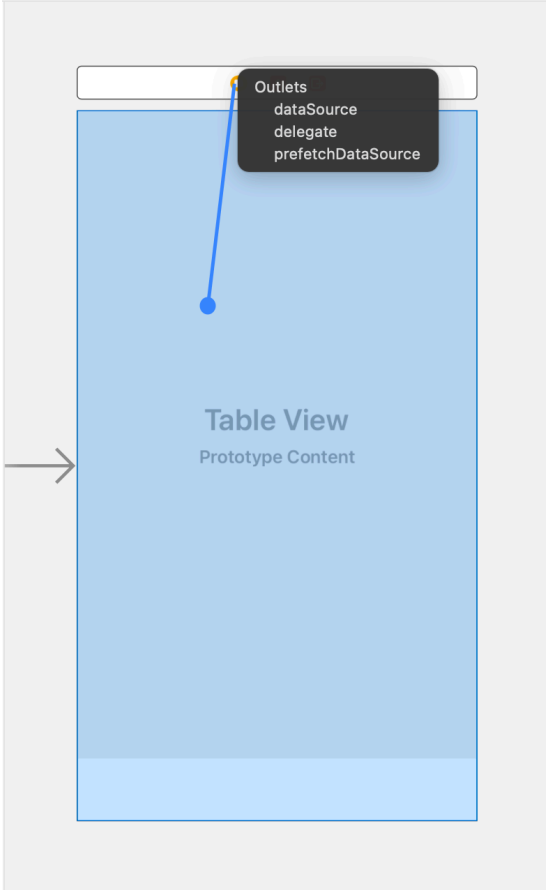


Figure 10 On ajoute les protocoles `Data Source` et `Delegate`.

Maintenant, sélectionnez à nouveau votre `Table View` et à droite de l'écran, dans `Attribute Inspector`, dans le champ `Prototype Cells`, remplacez le `0` par un `1` ; vous devriez voir apparaître une cellule dans le `Table View`.

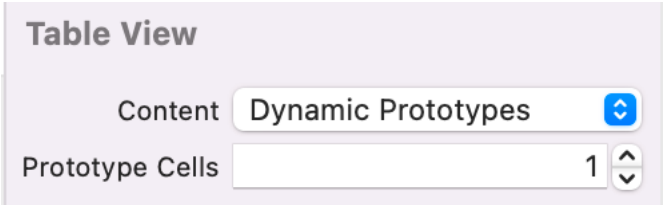


Figure 11On ajoute une cellule.

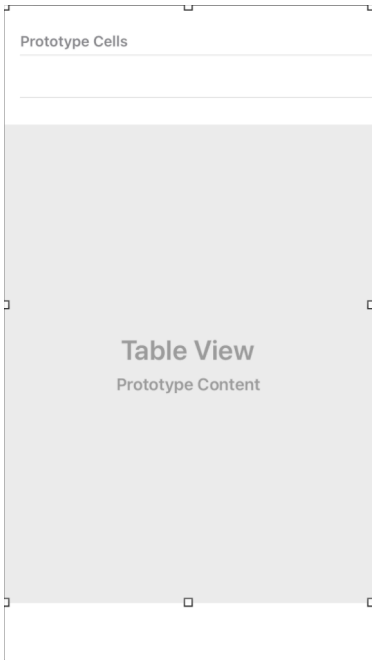


Figure 12Une cellule est apparue.

Sélectionnez cette cellule et dans `Attribute Inspector` (toujours à droite), dans le champ Identifier mettez l'indentifiant `celluleID`.

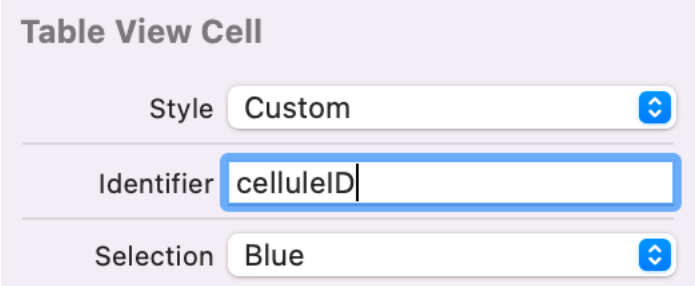


Figure 13On donne un identifiant à la cellule.

On passe maintenant en mode Assistant via un clic droit sur les petites lignes (symbolisant du code) ou via le menu Editor > Assistant.

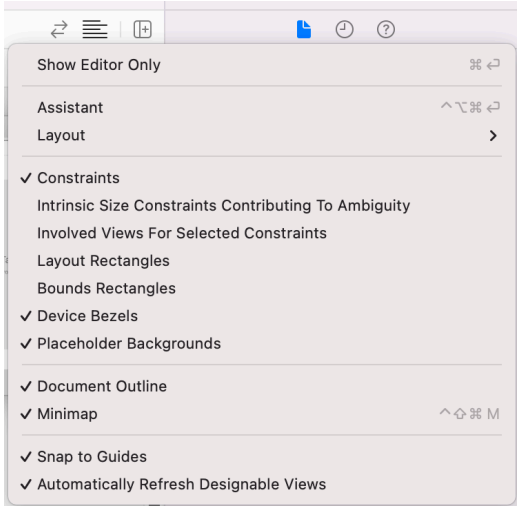


Figure 14On passe en mode assistant.

L'écran se scinde alors en deux, avec d'un côté le `Main.storyboard`, de l'autre le code.

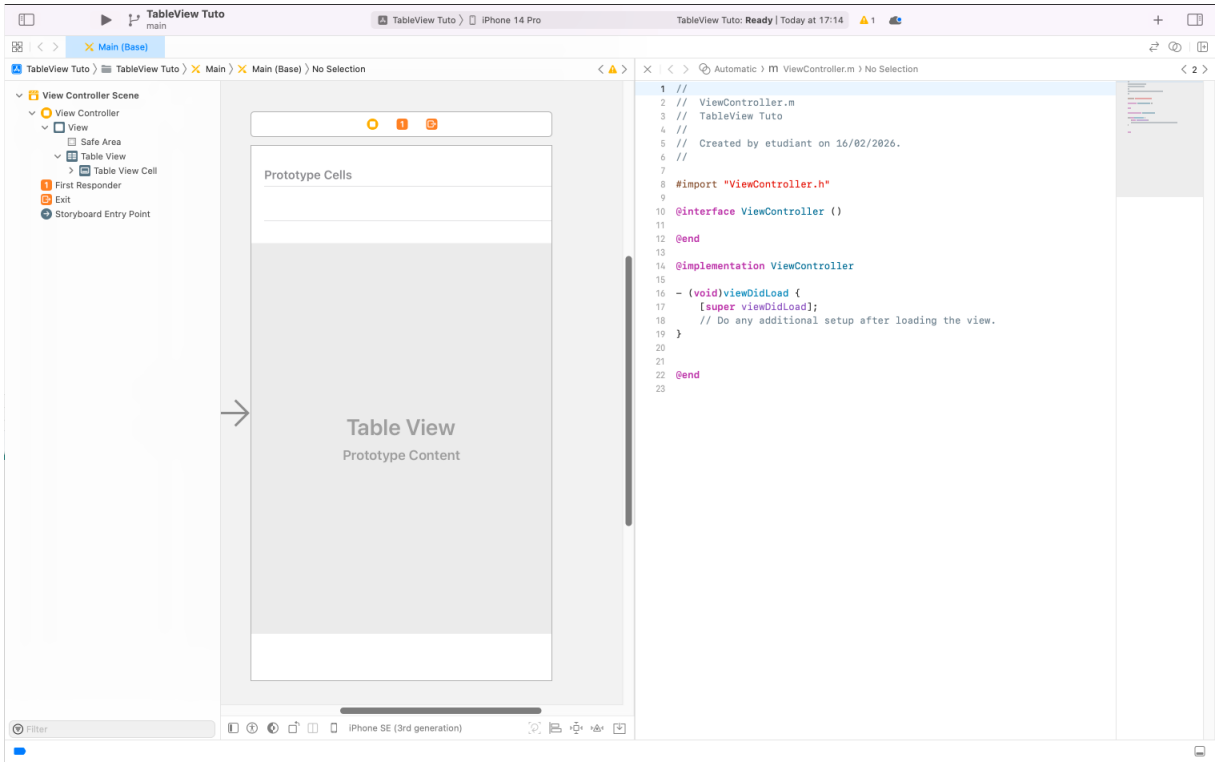
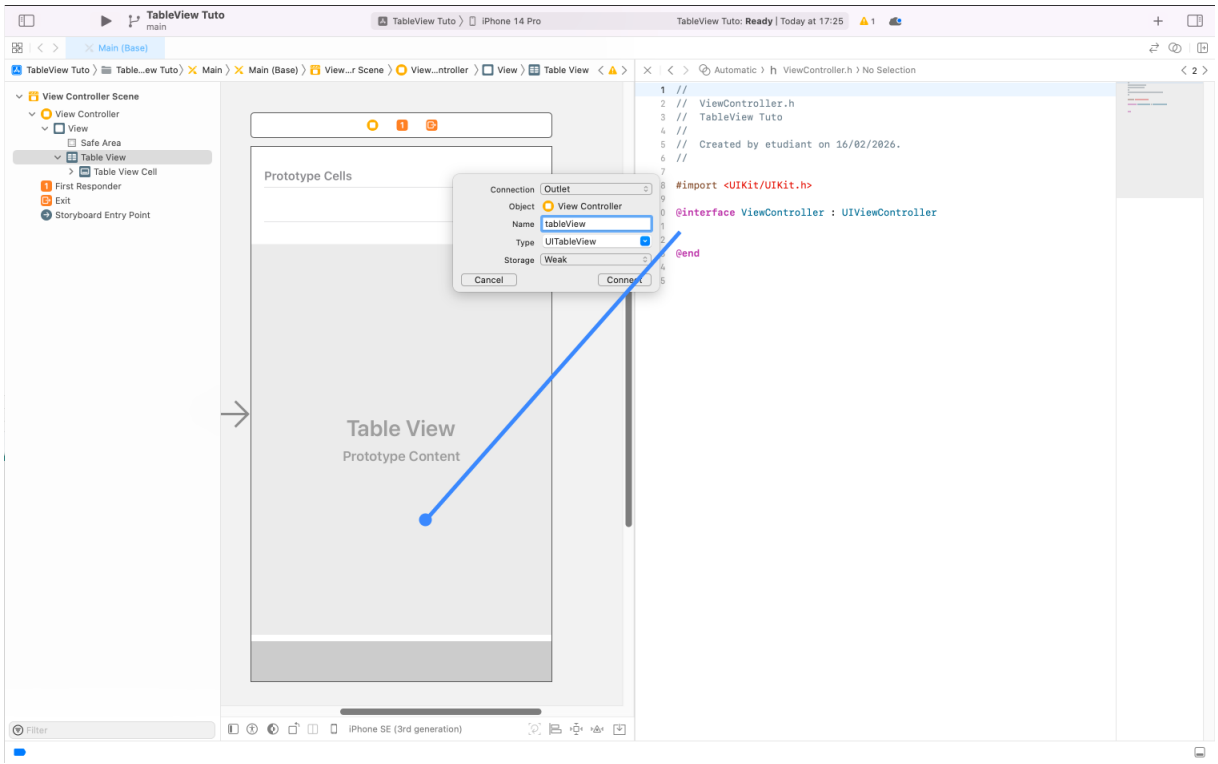


Figure 15Mode Assistant.

Dans la vue de droite, sélectionnez le fichier `ViewController.h`. Dans la vue de droite, sélectionnez votre `Table View`, maintenez la touche `control` et glissez le trait bleu jusque dans le `ViewController.h` au-dessus de `@end`. Donnez-lui un nom, par exemple `tableView` et cliquez sur `Connect`.



Quittez l'Assistant, pour revenir à une visualisation plus pratique pour coder.

Le code

Nous allons d'abord modifier le code de `ViewController.h` afin de le conformer aux deux protocoles `UITableViewDataSource` et `UITableViewDelegate` (rappelons que la notion de protocole en Objective-C est assez proche de celle d'interface en Java) et nous allons ajouter un attribut de type `NSArray` (un tableau à la mode Cocoa).

Programme 1 : Code de `ViewController.h`.

```
// ViewController.h
// TableView Tuto
//

#import <UIKit/UIKit.h>

@interface ViewController : UIViewController
                                        <UITableViewDataSource,
                                            UITableViewDelegate>
{
    NSArray *etudiants;
}
@property (weak, nonatomic) IBOutlet UITableView *tableView;

@end
```

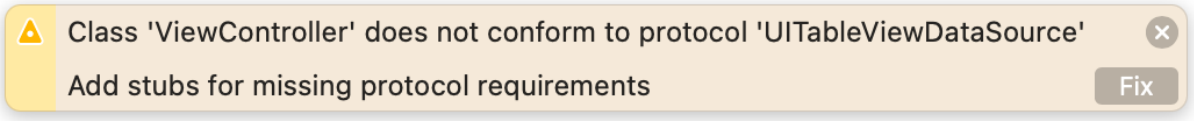
Maintenant passons à ViewController.m. C'est la méthode viewDidLoad qui va initialiser le tableau etudiant. On la modifie pour qu'elle devienne :

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
    etudiants = @[@"Pierre",@"Paul",@"Jacques"];
}
```

Il ne vous aura pas échappé qu'Xcode nous averti d'un problème :

```
Class 'ViewController' does not conform to protocol 'UITableViewDataSource'
```

C'est comme Java quand une interface n'implémente pas toutes les méthodes... eclipse nous propose de le faire pour nous. Ici c'est la même chose, XCode nous propose de nous aider en ajoutant ce qu'il manque :



Cliquez sur Fix et pas moins de 13 nouvelles méthodes apparaissent. Petite subtilité par rapports aux interfaces Java, les protocoles Objective-C proposent des méthodes qui peuvent être optionnelles. Ici, parmi les 13, seules 2 sont obligatoires :

```
tableView: numberOfRowsInSection:
tableView: cellForRowAtIndexPath:
```

Nous allons les implémenter. Ainsi le code de ViewController.m devient :

Programme 2 : Code de `ViewController.m`.

```
#import "ViewController.h"

@interface ViewController ()

@end

@implementation ViewController

- (void)viewDidLoad {

    [super viewDidLoad];

    // Do any additional setup after loading the view.

    etudiants = @[@"Pierre",@"Paul",@"Jacques"];

}
```

```
- (nonnull UITableViewCell *)
        tableView:(nonnull UITableView *)tableView
        cellForRowAtIndexPath:(nonnull NSIndexPath *)indexPath
{
     UITableViewCell *cell = [tableView
dequeueReusableCellWithIdentifier:@"celluleID"];
     cell.textLabel.text = etudiants[indexPath.row];
    return cell;
}

- (NSInteger)tableView:(nonnull UITableView *)tableView
numberOfRowsInSection:(NSInteger)section {
    return etudiants.count;
}

@end
```

Toujours plus loin

Faisons évoluer notre programme en lui ajoutant une fonctionnalité : l'arrivée d'un nouvel étudiant dans la liste quand on clique sur un bouton.

Tout d'abord on ajoute ce nouveau *widget* dans le `Main.storyboard`. On procède comme la dernière fois : on va chercher le widget Button dans la bibliothèque d'objet, on le place sous notre `TableView`, on passe en mode Assistant et par control-glisser vers `ViewController.m` on crée une méthode `reagir:` qui sera appelée quand on presse le bouton.

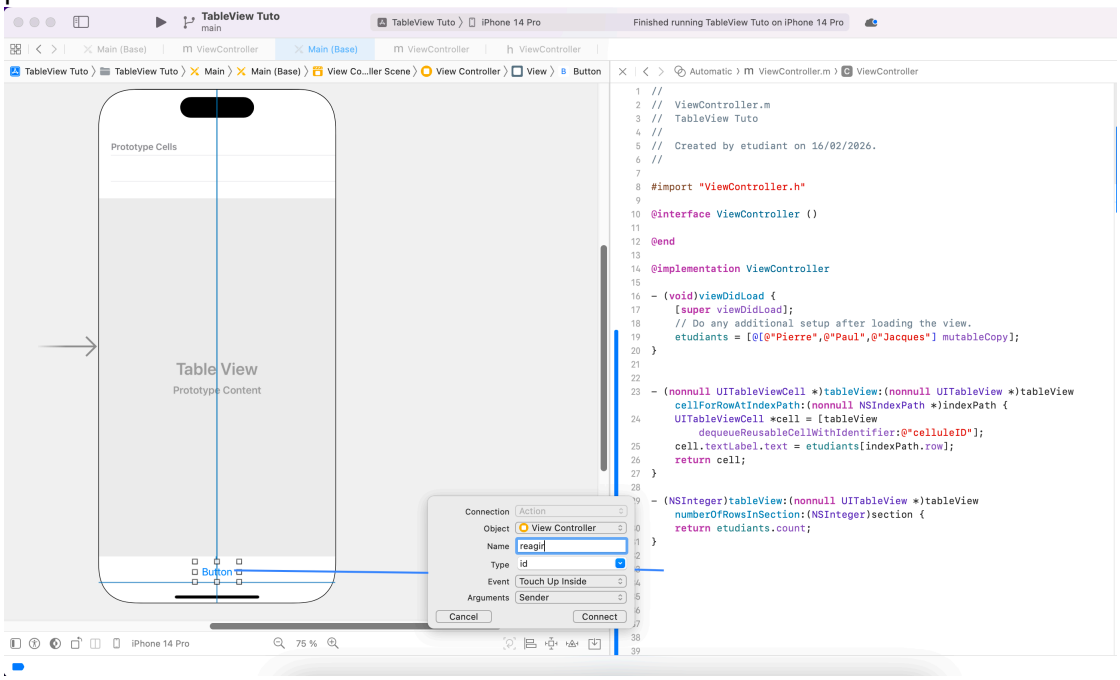


Figure 16Ajout d'un bouton.

Maintenant il nous faut modifier le code.

On veut que le tableau etudiant puisse accepter de nouveaux éléments. La classe NSArray définit des tableaux immuables (non *mutables*) il nous faut des tableaux *mutables* :

Programme 3 : Code de `ViewController.h`.

```
// ViewController.h
// TableView Tuto
//

#import <UIKit/UIKit.h>

@interface ViewController : UIViewController
                                        <UITableViewDataSource,
                                            UITableViewDelegate>
{
    NSMutableArray *etudiants;
}
@property (weak, nonatomic) IBOutlet UITableView *tableView;

@end
```

De même lors de l'initialisation de ce tableau on le transforme en `NSMutableArray` et la méthode viewDidLoad devient :

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
    etudiants = [@[@"Pierre",@"Paul",@"Jacques"] mutableCopy];
}
```

Il nous suffit maintenant d'implémenter la méthode `reagir:` avec le code suivant :

```
- (IBAction)reagir:(id)sender {
    [etudiants addObject:@"Karim"];  // on ajoute un objet
    [_tableView reloadData];         // on dit à tableView qu'il a
}                                    // de nouvelles données
```

Et voilà un `TableView` dynamique :

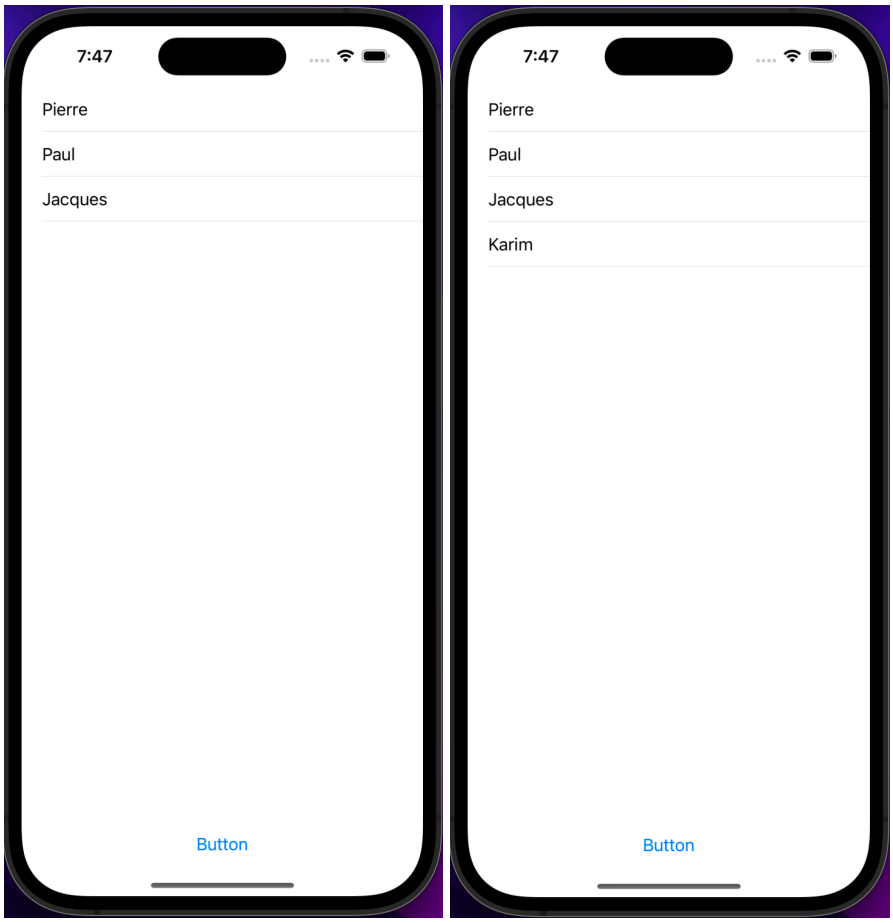


Figure 17 Avant et après l'appui sur le bouton.